4-Bit ALU using Verilog

Name: Gautam Raj Chhabra
Project: 4-Bit ALU Verilog
Language: Verilog HDL
Tool Used: EDA Playground (Icarus Verilog)

1. Abstract

This project implements a 4-bit Arithmetic Logic Unit (ALU) in Verilog HDL. The ALU performs different arithmetic and logical operations based on the opcode input. The design is verified using a testbench and simulated in EDA Playground.

2. Introduction

An Arithmetic Logic Unit (ALU) is a fundamental component of digital systems and processors. It performs arithmetic and logical operations on binary data. In this project, a 4-bit ALU is designed using behavioral modeling in Verilog. The ALU supports multiple operations such as addition, subtraction, AND, OR, NOT, shift left, and shift right.

3. Operations Supported

The ALU supports the following operations based on the opcode:

000 → Addition (A + B)
001 → Subtraction (A − B)
010 → AND (A & B)
011 → OR (A | B)
100 → XOR (A ^ B)
101 → NOT (~A)
110 → Left Shift (A << 1)
111 → Right Shift (A >> 1)

4.Verilog Design Code

```
module alu(input [3:0]a, input [3:0]b, input [2:0]op,
output reg [3:0] y);

  always@(*)
    begin
      case(op)

        3'b000 : y = a + b;
        3'b001 : y = a - b;
        3'b010 : y = a & b;
        3'b011 : y = a | b;
        3'b100 : y = a ^ b;
        3'b101 : y = ~a;
        3'b110 : y = a << 1;
        3'b111 : y = a >> 1;
        default: y=4'b0000;
      endcase
    end
  endmodule
```

5.Testbench

The testbench verifies the functionality of the 4-bit ALU by applying different values to inputs A, B, and opcode. A nested loop is used to test all possible combinations of inputs.

The $monitor command displays the input and output values during simulation, allowing verification of all arithmetic and logical operations.

This confirms that the ALU works correctly for all supported opcodes.

```
module testbench;
  reg [3:0]a;
  reg [3:0]b;
  reg [2:0]op;
```

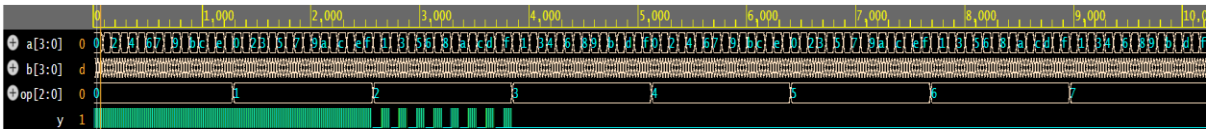
```
  wire [3:0]y;
  integer i,j,k;


  alu DUT(a,b,op,y);
  initial
    begin
      $monitor($time,"a=%b,b=%b,op=%b,y=%b",a,b,op,y);


      for(k=0; k<8 ; k=k+1)begin
        op=k;
        for(i = 0; i < 16; i = i + 1) begin
        for(j = 0; j < 16; j = j + 1) begin

            a = i;
            b = j;
          #5;
        end
        end
      end
      $finish;
    end
endmodule
```

6. Simulation Result

The loop limits in the testbench were reduced to obtain a shorter simulation output for easier verification of the ALU operations.

```
[2026-03-07 07:45:50 UTC] iverilog '-Wall' '-g2012' design.sv testbench.sv  && unbuffer vvp a.out
VCD info: dumpfile dump.vcd opened for output.
                  0a=0000,b=0000,op=000,y=0
                  5a=0000,b=0001,op=000,y=1
                 10a=0001,b=0000,op=000,y=1
                 15a=0001,b=0001,op=000,y=0
                 20a=0000,b=0000,op=001,y=0
                 25a=0000,b=0001,op=001,y=1
                 30a=0001,b=0000,op=001,y=1
                 35a=0001,b=0001,op=001,y=0
                 40a=0000,b=0000,op=010,y=0
                 45a=0000,b=0001,op=010,y=0
                 50a=0001,b=0000,op=010,y=0
                 55a=0001,b=0001,op=010,y=1
                 60a=0000,b=0000,op=011,y=0
                 65a=0000,b=0001,op=011,y=0
                 70a=0001,b=0000,op=011,y=0
                 75a=0001,b=0001,op=011,y=0
testbench.sv:30: $finish called at 80 (1s)
Done
```

7. EP Wave



8. Conclusion

The 4-bit ALU was successfully designed using Verilog HDL and tested using a testbench. Simulation results and EPWave waveform confirm the correct functionality of all supported operations. This project shows the use of digital design and verification using Verilog.